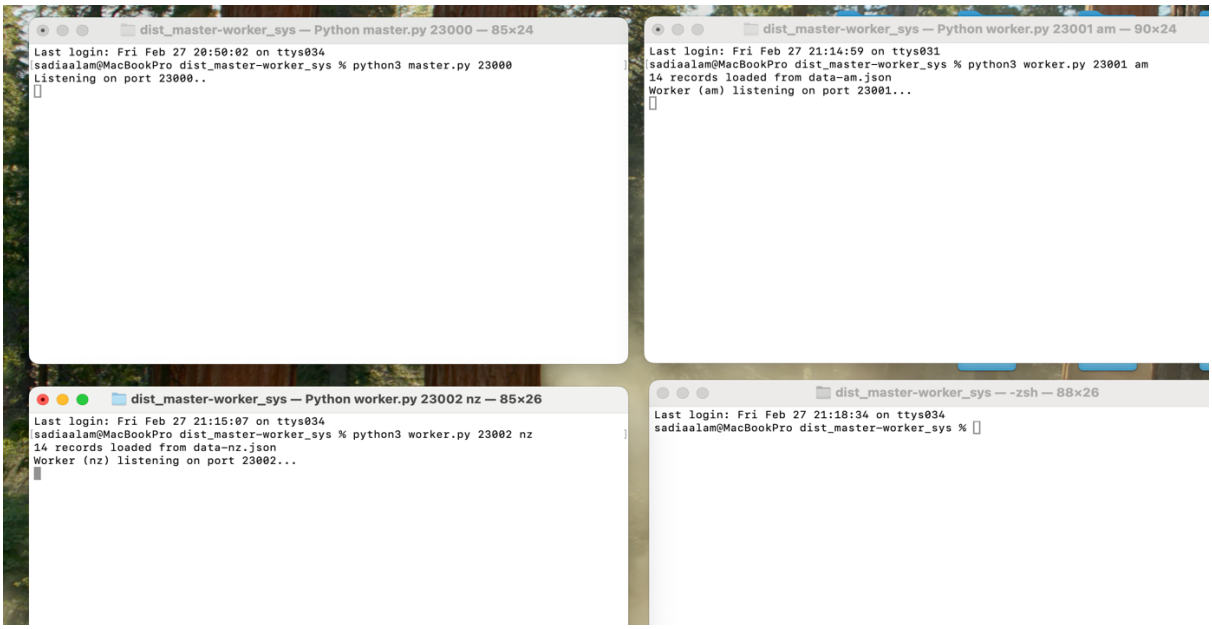
Sadia Alam

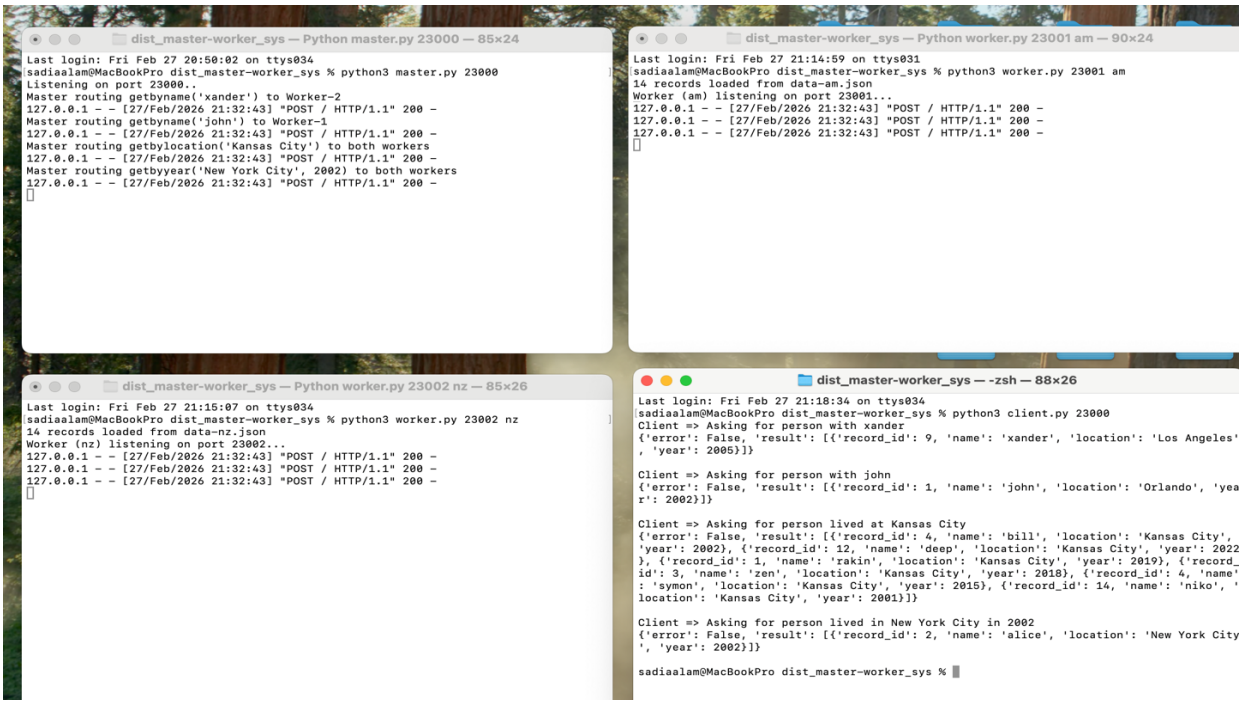
CS446: Distributed Computing System

02/27/2026

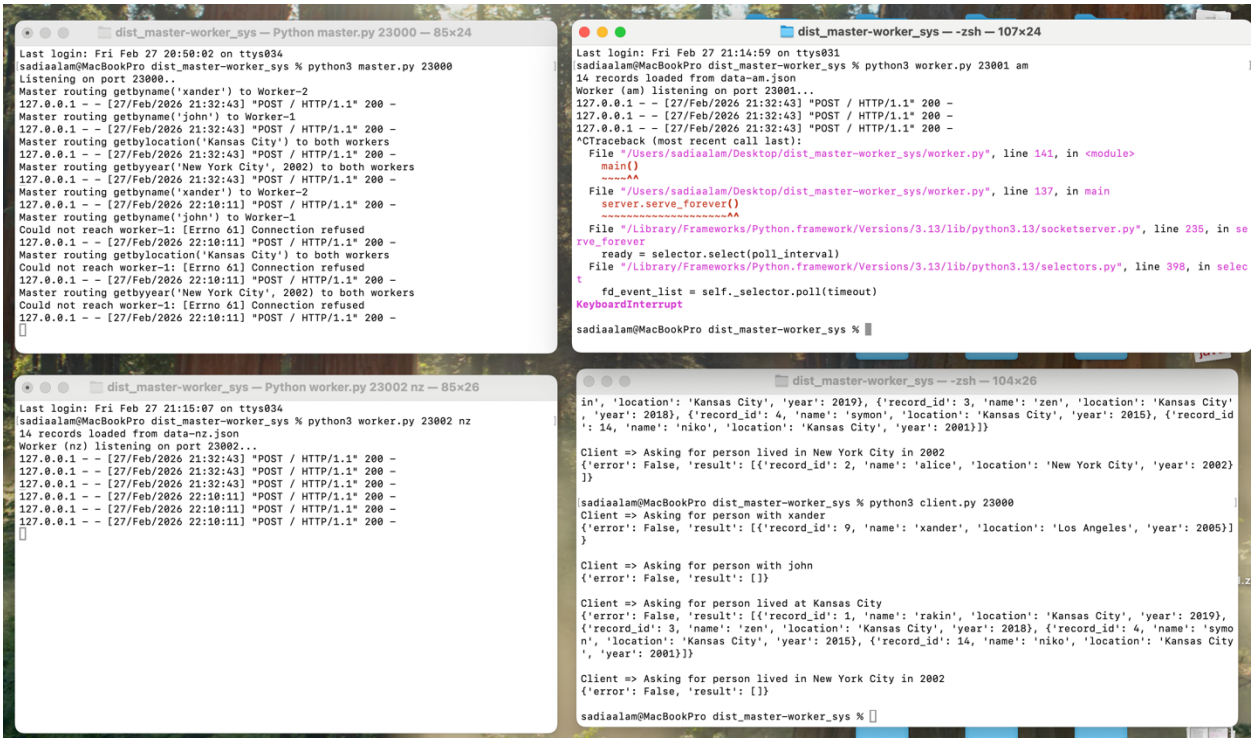
Distributed Master-Worker System



The master and worker servers are running successfully. The master and worker processes are listening to port 23000, 23001 and 23002 respectively. The worker processes load their relevant JSON file into python dictionary object.



When the client program is executed, master receives client's request and passes it to the correct worker and then forwards the matching records from worker process to the client. For example, in our program, the client calls the getbylocation function for a person's info who lives in Kansas City. The worker servers find and return the person's records that matches the location from their loaded JSON file. The master program processes the result received from the workers and sends it to the client. Thus, we see the records of the persons who are residents of Kansas City in the client terminal.



From the screenshot, we can see that I killed the worker 1 server. When the client program is executed, the master can't send request to worker 1 but sends the request to worker 2. Therefore, in the client terminal, we see the matching records the master routed from worker 2.

It's quite interesting that even if one of the servers are killed, a failed connection doesn't crash our whole program. When one server is killed the master server stays alive. Master server works as a middleman here. When one worker server is down, it sends the request to the other active server and returns whatever results it could get from the running worker. For example, in our program, the client requests for person's records with the name 'John'. But the worker 1 server handles the info for this request. Instead of crashing our program it returned an empty list. The master process doesn't store any data. It connects between the client and worker processes. If the master is down, then no requests can be processed even if the worker severs are active. Also, I noticed how the work is evenly balanced between the worker processes. If worker 1 is killed, the queries with the names that start with letter n-z don't have to suffer. For example, in our program, when the client requests for the records of the residence of Kansas

City the query is sent to both workers. Since worker 1 is down, the matching records from worker2 gets displayed.